Technical Project Report: Student Management System (SMS)

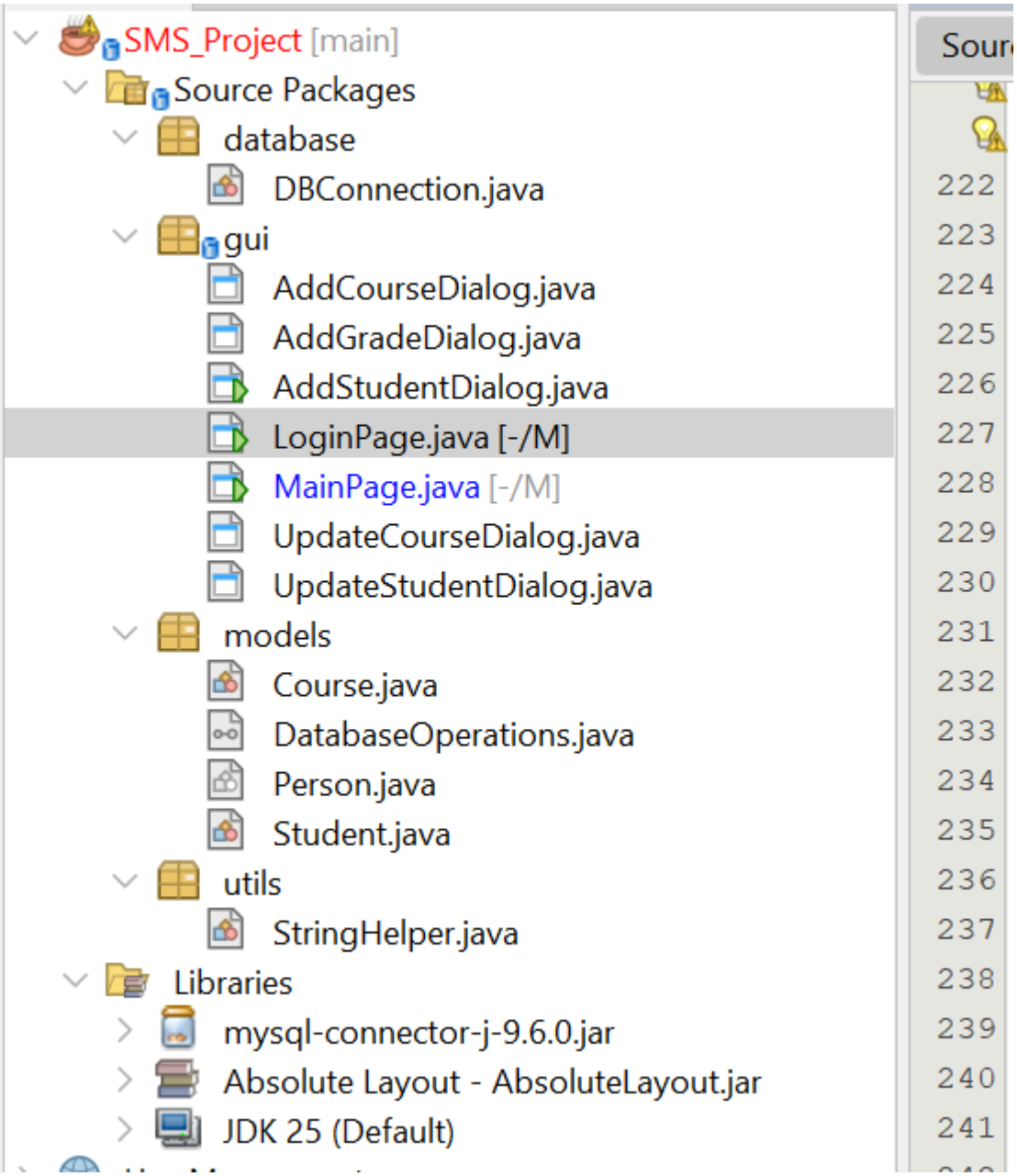
Developer: buster_call_Root **Institution:** University of Rwanda, College of Science and Technology **Project:** Java Swing Desktop Application with MySQL Integration

1. Introduction

The Student Management System (SMS) is a desktop-based graphical application designed to streamline the administration of university records. Built using Java Swing and connected to a MySQL database, this system allows administrators to perform seamless Create, Read, Update, and Delete (CRUD) operations on both student records and academic courses. The primary objective of this project was to build a secure, robust, and user-friendly system utilizing enterprise software engineering principles.

2. System Architecture

The application was designed using a strict **Model-View-Controller (MVC)** architectural pattern to separate the backend database logic from the frontend graphical user interface.



Caption: Figure 1 - Project package structure in NetBeans.

The project is divided into distinct, modular packages:

- **`gui` (Views/Controllers):** Contains all `JFrame` and `JDialog` classes. It handles user interactions, input validation, and dynamic UI updates (e.g., `MainPage.java`, `LoginPage.java`).
- **`models` (Models):** Contains the Data Transfer Objects (DTOs) and the core SQL logic (`Student.java`, `Course.java`). These classes interact directly with the database.
- **`database`:** Houses the `DBConnection.java` file, which manages the secure connection to the local MySQL server.

- **`utils`:** Contains helper classes like `StringHelper.java` for password hashing and data sanitization.

3. Database Design

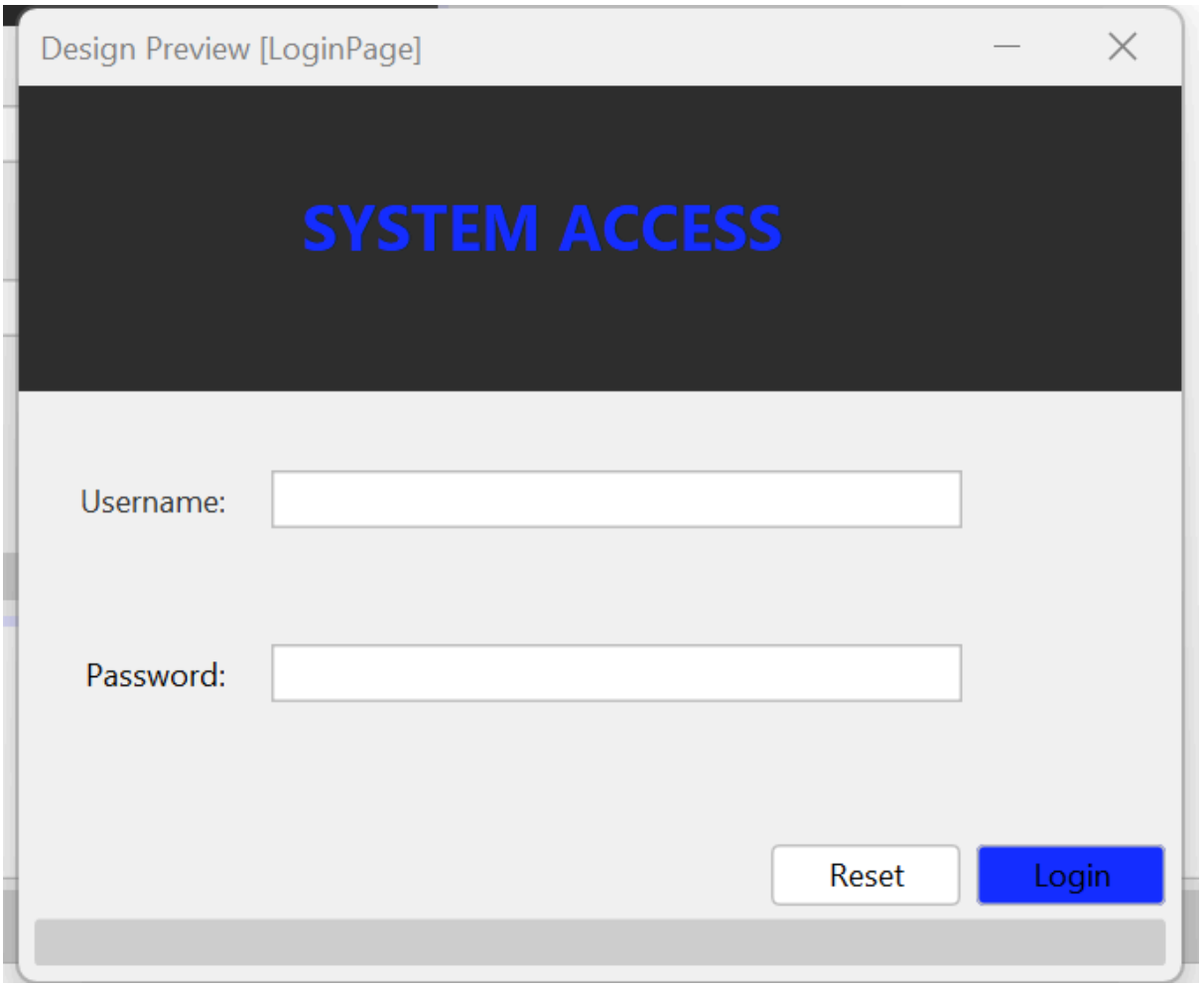
The system relies on a normalized relational database containing three primary tables:

1. **`students`:** Stores student demographic information (ID, Name, Email).
2. **`courses`:** Stores academic courses (Course ID, Course Name).
3. **`enrollments`:** Acts as a junction table connecting students to courses and storing their specific grades/marks.

4. Key Features & User Interface

4.1 Secure Authentication

The application is protected by a login screen that requires valid credentials before accessing sensitive university data.



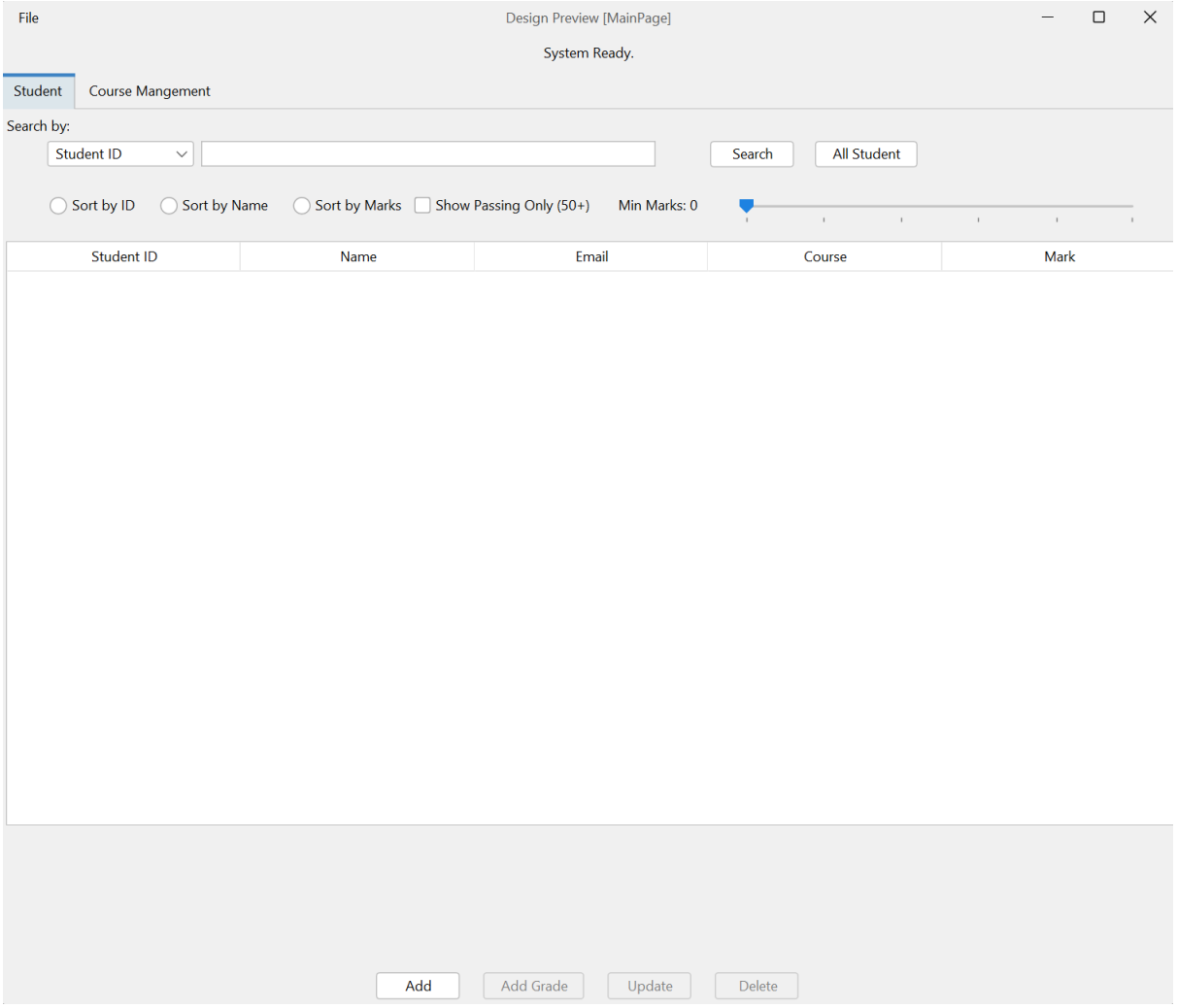
```java
private void loginBtnActionPerformed(java.awt.event.ActionEvent evt) {

    // 1. Grab the RAW data first
    String user = usernameField.getText().trim();
    char[] rawPassword = passwordField.getPassword();

    // 2. VALIDATE FIRST! (Before you lock the UI)
    if (user.isEmpty() || rawPassword.length == 0) {...7 lines }

    // 3. Now that we know it's not empty, it is safe to hash it
    String hashPass = utils.StringHelper.hashPassword(rawPassword);

    // 4. Lock the UI and start animations
    loginProgressBar.setIndeterminate(true);
    loginBtn.setEnabled(false);
    messageLabel.setText("Logging in...");

    // 5. Create the SwingWorker to do the heavy lifting in the background
    javax.swing.SwingWorker<Boolean, Void> worker = new javax.swing.SwingWorker<Boolean, Void>() {...54 lines };

    // 6. Actually start the background worker!
    worker.execute();
```

Caption: Figure 2 - The secure login interface.

- **Technical Detail:** The login process utilizes a `SwingWorker` thread. This ensures that the database authentication query runs in the background, allowing the UI to display a smooth loading progress bar without freezing the application.

4.2 The Master Dashboard (Context-Aware Navigation)

The core of the application is a single master dashboard utilizing a `JTabbedPane`.



Caption: Figure 3 - The main dashboard displaying student records.

- **Technical Detail:** Instead of duplicating action buttons, the system uses a "Master Action Panel" at the bottom of the screen. The "Add", "Update", and "Delete" buttons are context-aware; they use action listeners to determine which tab the user is currently viewing and instantly adapt their functionality to either manage Students or Courses.

4.3 Advanced Searching and Filtering

```java
private void refreshTable() {
    int activeTab = myTabbedPane.getSelectedIndex();

        if (activeTab == 0) {
            // ==========================================
            //            STUDENT TAB LOGIC
            // ==========================================
            String keyword = searchField.getText().trim();

            String sortBy = "ID";
            if (sortNameRadio.isSelected()) sortBy = "NAME";
            else if (sortMarksRadio.isSelected()) sortBy = "MARKS";

            double minMarks = passingCheckBox.isSelected() ? 50.0 : marksSlider.getValue();

            // Ask Student model for data
            java.util.ArrayList<Object[]> data = models.Student.getData(keyword, sortBy, minMarks);

            // Paint Student Table
            javax.swing.table.DefaultTableModel model = (javax.swing.table.DefaultTableModel) studentTable.getModel();
            model.setRowCount(0);
            for (Object[] row : data) {
                model.addRow(row);
            }

        } else if (activeTab == 1) {
```

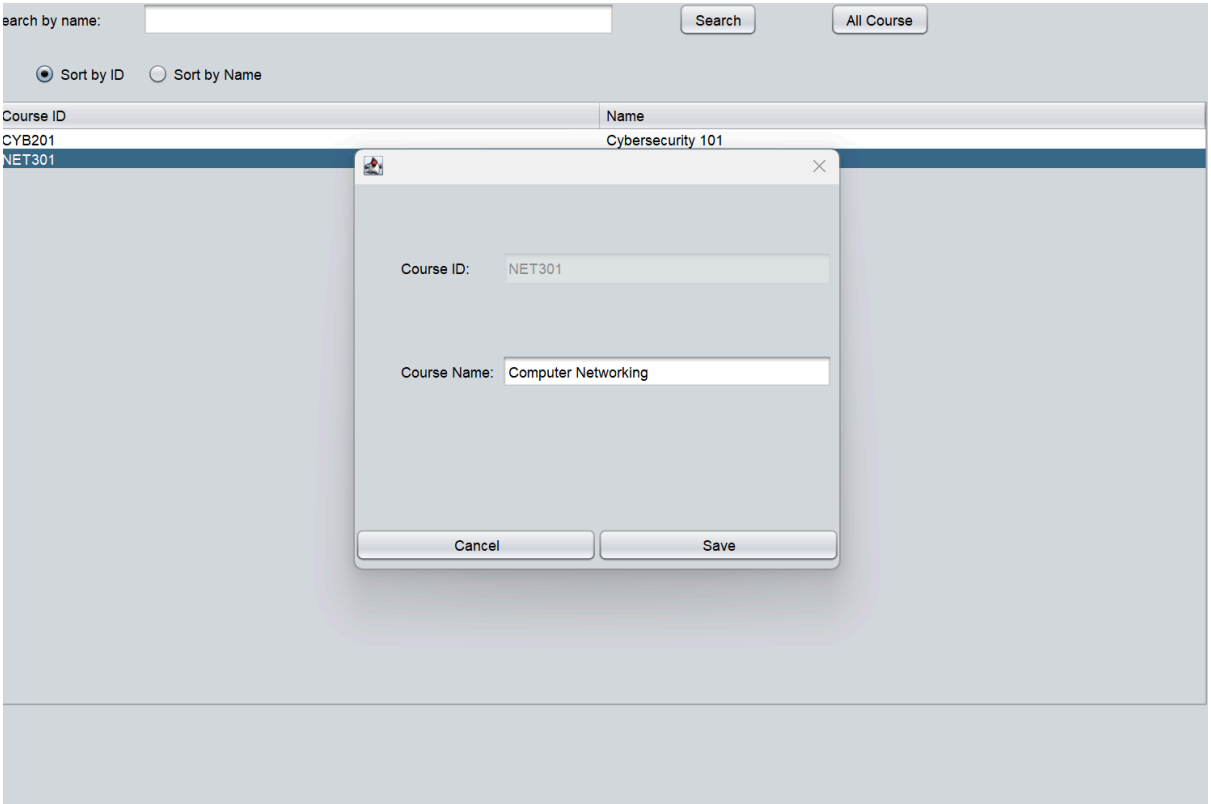
```java
        } else if (activeTab == 1) {
            // ==========================================
            //            COURSE TAB LOGIC
            // ==========================================
            String keyword = searchField1.getText().trim();
            boolean sortByName = sortNameRadio1.isSelected();

            // Ask Course model for data
            java.util.ArrayList<String[]> data = models.Course.getData(keyword, sortByName);

            // Paint Course Table
            javax.swing.table.DefaultTableModel model = (javax.swing.table.DefaultTableModel) courseTable.getModel();
            model.setRowCount(0);
            for (String[] row : data) {
                model.addRow(row);
            }
        }
```

Caption: Figure 4 - Real-time searching and sorting functionality.

- **Technical Detail:** The UI features real-time search capabilities. When a user enters a keyword, the system dynamically generates a SQL `LIKE` query (e.g., `LIKE '%keyword%'`) to filter the `JTable` instantly. Users can also sort data by Name or ID using connected Radio Buttons.

4.4 Defensive UI and Data Sanitization

To maintain database integrity, the system utilizes "Defensive Programming" techniques inside its data-entry pop-ups.

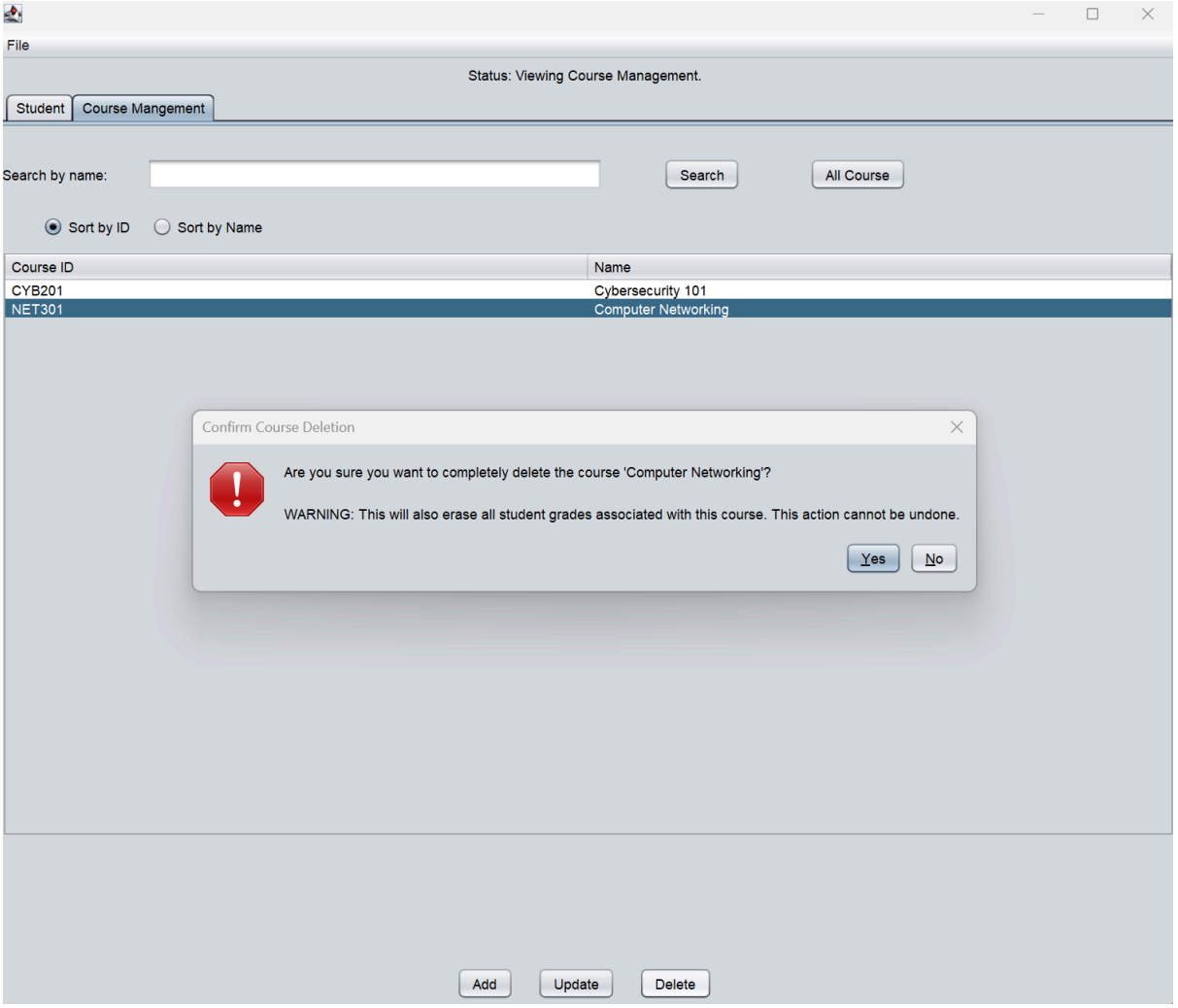


Caption: Figure 5 - Course management dialog with locked primary keys.

- **Technical Detail:** When updating a course, the `Course ID` text field is disabled (`setEditable(false)`) to strictly enforce the rule that Primary Keys cannot be altered. Additionally, all user inputs are sanitized before being sent to the database (e.g., automatically converting "intro to java" to Title Case and trimming invisible whitespace).

4.5 Safe Deletion via SQL Transactions

One of the most critical features is the secure deletion process.



Caption: Figure 6 - Confirmation dialog warning the user about cascading deletions.

- **Technical Detail:** Because deleting a course would cause Foreign Key integrity errors if students are still enrolled, the `Course.delete()` method utilizes a strict **SQL Transaction** (`conn.setAutoCommit(false)`). It first safely drops the associated grades from the `enrollments` table, then deletes the course, and finally commits the changes. If either step fails, the entire transaction rolls back to protect the data.

5. Conclusion

Developing the Student Management System provided extensive hands-on experience with object-oriented programming, graphical interface design, and relational database management. By successfully implementing the MVC architecture, background threading (`SwingWorker`), and defensive UI logic, the resulting application is highly robust, scalable, and ready for real-world deployment.